

EmPay

Smart Human Resource Management System

Software Requirements Specification (SRS)

Version 1.0 | Hackathon Project | Odoo Challenge

Date: May 2026

Team Member	Role
Member 1 (Vismay Rathod)	Frontend Developer
Member 2 (Kishan Kansara)	Backend Developer
Member 3 (Manav Vadhavana)	Backend Developer
Member 4 (Mohak Vyas)	Project Explainer / Client Interaction

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional and nonfunctional requirements for EmPay — a Smart Human Resource Management System (HRMS). It serves as the primary reference for all four team members during development, design, and client demonstration phases.

1.2 Project Scope

EmPay is an all-in-one HRMS platform targeted at startups, institutions, and SMEs. It consolidates user management, attendance tracking, leave workflows, payroll processing, and analytics into a single web interface.

1.3 Definitions & Terminologies

Term	Definition
Payroll	Process of calculating and distributing employee salaries, wages, bonuses, and deductions based on attendance records.

Payrun	A specific payroll cycle/period during which salaries are processed based on attendance, approved leaves, and deductions.
Payslip	Official document per payrun showing a detailed breakdown of earnings, deductions, and net pay.
Time-Off	Approved absence period (vacation, sick, personal leave) factored into payroll calculations.
Wage	Monetary compensation calculated based on attendance, working hours, and approved leaves.
PF Contribution	Both employee and employer contribute 12% of basic salary monthly as long-term retirement savings.
Professional Tax	Monthly state-levied tax deducted from gross salary during payroll processing.
HRMS	Human Resource Management System — the complete platform being built.
RBAC	Role-Based Access Control — restricting system access based on user roles.

1.4 References

- EmPay – Smart Human Resource Management System Problem Statement PDF (Odoo Hackathon)
- Mockup: <https://link.excalidraw.com//65VNwvy7c4X/7gx0B8JymIS>

2. Overall System Description

2.1 System Overview

EmPay provides a clean, role-gated web application where four distinct user types interact with appropriate modules. The system enforces RBAC at every layer — from UI navigation to API endpoints — ensuring data integrity and privacy.

2.2 User Roles & Responsibilities

Admin — Full Access

- Register on the portal and manage all user accounts.
- Create, Read, Update, Delete data across all modules.
- Manage user roles via Settings panel.
- Oversee all system activities; no access limitations.

HR Officer — People & Attendance

- Create and update employee profiles and personal details.
- Monitor and view attendance records of all employees.
- Manage and allocate leaves to employees.
- Cannot access payroll data or system settings.

Payroll Officer — Compensation & Reports

- Approve or reject employee time-off requests.
- Generate payslips and leave reports.
- Manage payroll, time-off, and reporting modules.
- View attendance records (read-only).
- Cannot create/modify employee data or access system settings (except salary info).

Employee — Self-Service

- Mark daily attendance (check-in / check-out).
- Apply for time off and view leave status.
- View personal attendance and performance records.
- Access employee directory in read-only mode.
- Cannot access settings, payroll, reports, or salary info of others.

2.3 Role Access Matrix

Module / Feature	Admin	HR Officer	Payroll Officer	Employee
User Registration & Login	✓ Full	✓ Full	✓ Full	✓ Full
Profile Management	✓ Full	✓ All Employees	✗ None	✓ Own Only
Employee Directory	✓ Full	✓ Full	✓ Read	✓ Read
Attendance — Mark	✓ Full	✓ Full	✓ View Only	✓ Own Only
Attendance — View All	✓ Full	✓ Full	✓ Full	✗ None
Leave — Apply	✓ Full	✓ Full	✓ Full	✓ Own Only
Leave — Approve/Reject	✓ Full	✓ Full	✓ Full	✗ None
Leave — Allocate	✓ Full	✓ Full	✗ None	✗ None
Payroll — View	✓ Full	✗ None	✓ Full	✓ Own Payslip

Payroll — Generate/Edit	✔ Full	✗ None	✔ Full	✗ None
Reports	✔ Full	✗ None	✔ Full	✗ None
Settings / Role Mgmt	✔ Full	✗ None	✗ None	✗ None
Dashboard & Analytics	✔ Full	✔ Partial	✔ Partial	✔ Own Data

3. Functional Requirements

3.1 Module 1 — User & Role Management

3.1.1 User Registration & Login

- FR-URM-01: The system shall allow new users to register with name, email, password, and role selection.
- FR-URM-02: The system shall validate email uniqueness and enforce password strength rules.
- FR-URM-03: Users shall be able to log in with email/password credentials.
- FR-URM-04: The system shall support session management with JWT/cookie-based auth.
- FR-URM-05: Admin can activate, deactivate, or delete user accounts.

3.1.2 Role-Based Access Control

- FR-URM-06: Every route and API endpoint must validate the authenticated user's role before granting access.
- FR-URM-07: Admin can change any user's role from the Settings panel.
- FR-URM-08: UI navigation menus shall dynamically render only role-permitted modules.

3.1.3 Profile Management

- FR-URM-09: Employees can edit their own profile (name, contact, profile picture).
- FR-URM-10: HR Officers can edit any employee's profile details.
- FR-URM-11: Admins can edit all fields including role and salary info.

3.2 Module 2 — Attendance & Leave Management

3.2.1 Attendance Tracking

- FR-ATT-01: Employees shall be able to mark Check-In and Check-Out once per working day.
- FR-ATT-02: The system shall auto-calculate daily working hours from check-in/check-out timestamps.
- FR-ATT-03: Employees can view their own daily and monthly attendance logs.
- FR-ATT-04: HR Officers and Admin can view attendance of all employees.
- FR-ATT-05: Attendance status options: Present, Absent, Half-Day, On Leave, Holiday.

3.2.2 Leave Management

- FR-LVE-01: Employees shall submit leave applications specifying type (Sick / Casual / Annual / Unpaid), start date, end date, and reason.
- FR-LVE-02: Leave status workflow: Pending → Approved / Rejected.
- FR-LVE-03: Payroll Officers and Admin can approve or reject leave applications.
- FR-LVE-04: HR Officers can allocate leave balances (e.g., 12 annual, 6 sick) to employees.
- FR-LVE-05: Employees can view remaining leave balances per type.
- FR-LVE-06: Approved leaves shall be automatically factored into payroll calculations.

3.3 Module 3 — Payroll Management

3.3.1 Payroll Processing

- FR-PAY-01: Payroll is calculated per monthly payrun based on attendance, approved leaves, and applicable deductions.
- FR-PAY-02: Salary breakdown must display: Basic Salary, HRA, Allowances, Gross Salary, PF Deduction (12% of Basic), Professional Tax, Other Deductions, Net Pay.
- FR-PAY-03: Payroll Officers and Admin can generate monthly payroll for all employees.
- FR-PAY-04: Admin/Payroll Officer can edit payroll entries before finalizing.
- FR-PAY-05: Once finalized, payslips are auto-generated and locked from editing.

3.3.2 Payslips & Reports

- FR-PAY-06: Each employee shall be able to view and download their own payslip per payrun.
- FR-PAY-07: Payroll Officers/Admin can generate bulk leave and payroll reports.
- FR-PAY-08: Reports shall be exportable (PDF format minimum).

3.4 Module 4 — Dashboard & Analytics

- FR-DSH-01: Admin dashboard shows: total employees, attendance summary, pending leave requests, payroll status, recent activities.
- FR-DSH-02: Charts to display: monthly attendance trends (bar/line), leave distribution (pie), payroll cost over months (bar).
- FR-DSH-03: Employee dashboard shows personal attendance summary, leave balances, and latest payslip.
- FR-DSH-04: HR Officer dashboard shows team attendance overview and pending leave requests.
- FR-DSH-05: Payroll Officer dashboard shows payruns pending processing and recent report activity.

4. Non-Functional Requirements

Category	Requirement	Description
Performance	Response Time	API responses must complete within 2 seconds under normal load.
Security	Authentication	JWT-based auth with token expiry; passwords hashed with bcrypt.
Security	Authorization	Every API endpoint validates role before processing request.
Scalability	Architecture	Modular backend allowing independent scaling of payroll/attendance services.

Usability	Responsive UI	Works on desktop and tablet screens; minimum 768px viewport.
Reliability	Uptime	System should handle concurrent users without data corruption.
Maintainability	Code Quality	Meaningful Git commits; modular code structure; documented APIs.
Data Integrity	Payroll Lock	Finalized payslips cannot be edited; audit trail maintained.

5. Team Work Division

The project is divided among 4 team members. Each person owns a specific domain with clear deliverables. Cross-communication happens via Git PRs and daily standups.

Member 1 — Frontend Developer [Vismay Rathod]

Primary Responsibility: Build all user-facing screens, ensure responsive design, implement rolebased UI rendering.

Ownership & Deliverables:

- Login / Registration pages with form validation.
- Role-aware navigation sidebar (dynamically shows only permitted modules).
- Employee profile page (view + edit mode).
- Attendance module UI: Check-in/out button, daily log table, monthly calendar view.
- Leave module UI: Application form, leave status tracker, leave balance cards.
- Payroll UI: Salary breakdown table, payslip view, payslip download button.
- Admin & role-specific dashboards with Chart.js / Recharts visualizations.
- Reports page (generate/export buttons).
- Settings page (role management, system config — Admin only).
- Responsive design and consistent component library (React/Next.js + Tailwind). **API**

Integration:

- Consume REST APIs provided by Backend Dev; handle loading/error states.
- Implement token storage and attach Authorization headers on every request.
- Handle 401/403 redirects gracefully.

Tech Stack: React.js / Vite, Tailwind CSS, Chart.js or Recharts, React Query, excel.

Member 2 — Backend Developer (Lead) [Kishan Kansara]

Primary Responsibility: Core API design, database architecture, authentication system, and business logic for all primary modules.

Ownership & Deliverables:

- Database schema design: Users, Roles, Employees, Attendance, Leaves, Payroll, Payslips tables with relationships.
- Auth system: Registration, Login, JWT token generation, refresh, logout.
- RBAC middleware: Role-guard on every API route.
- User Management APIs: CRUD operations for user accounts and role updates.
- Employee Profile APIs: GET, PUT endpoints for profile data.
- Attendance APIs: Mark check-in/check-out, fetch daily/monthly logs per employee.
- Leave APIs: Apply, view, approve/reject, allocate leave balances.
- Payroll engine: Calculate gross, deductions (PF = 12% basic, Prof. Tax), net pay per employee per payrun.
- Payslip APIs: Generate, fetch, lock after finalization.
- API documentation (Postman collection or Swagger).

Tech Stack: Node.js + Express.js (FastAPI), MySQL, ORM, JWT.

Member 3 — Backend Developer [Manav Vadhavana]

Primary Responsibility: Support backend with reports/analytics APIs, PDF generation, data seeding, testing, and DevOps setup.

Ownership & Deliverables:

- Dashboard & Analytics APIs: Aggregate attendance stats, leave summaries, payroll cost trends for charts.
- Reports generation: Monthly payroll report, leave report — output as PDF (using PDFKit / Puppeteer).
- Payslip PDF export endpoint: Auto-render payslip template to downloadable PDF.
- Data seeding scripts: Populate DB with realistic demo data for all roles for demo purposes.
- Unit & integration testing: Write test cases for core payroll calculation logic.
- Environment setup: Docker-compose (optional), .env configuration, README with setup instructions.
- Git repository management: Enforce branching strategy (main / dev / feature branches), PR reviews.
- Support Backend Lead with any overflow API tasks (e.g., pagination, filtering, search). **Tech Stack:** Node.js, PDFKit, openPyExcel, Report Lab for payroll pdf generation.

Member 4 — Project Explainer / Client Interaction Specialist

[Mohak Vyas]

Primary Responsibility: Own the SRS document, prepare all demo materials, manage client communication, and lead the final presentation/demo walkthrough.

Ownership & Deliverables:

- Own and maintain this SRS document (update on scope changes).
- Create and maintain the project presentation deck (problem to solution to demo to architecture).
- Prepare demo script: flow through all 4 roles showing key workflows end-to-end.
- Write README.md: project overview, setup steps, credentials for demo accounts (all 4 roles).
- Conduct internal UAT (User Acceptance Testing) walkthroughs before final demo.
- Lead the hackathon demo: explain the system to judges/clients confidently.
- Collect feedback from judges and document improvement notes.
- Create a short Architecture Diagram showing module interactions (Employee to Attendance to Payroll).
- Prepare FAQ document: anticipated judge questions with prepared answers.
- Coordinate timeline across all team members; track task completion.

Tools: Google Slides / PowerPoint, GitHub Projects (task tracking).

6. System Architecture & Module Interaction

6.1 High-Level Architecture

Architecture: 3-Tier Web Application

Layer 1 — Frontend (React/Next.js): Role-aware UI, API consumer, data visualization.

Layer 2 — Backend API (Node.js/Express or Django): Business logic, RBAC enforcement, REST endpoints.

Layer 3 — Database (PostgreSQL/MySQL): Persistent storage for all entities.

6.2 Core Module Interaction Flow

Module Dependency Chain (as defined in problem statement)

Employees → Attendance → Payroll

Step 1: Employee records must exist before attendance can be tracked.

Step 2: Attendance records for the pay period are fetched when a Payrun is initiated.

Step 3: Approved Leaves are fetched and used to adjust working days count.

Step 4: Payroll engine calculates: Gross Pay, PF (12% Basic), Prof. Tax, Net Pay. Step 5: Payslips are auto-generated and locked post-finalization.

6.3 Database Tables Overview

Table	Key Fields
users	id, name, email, password_hash, role, is_active, created_at
employees	id, user_id (FK), department, designation, join_date, basic_salary, phone, address
attendance	id, employee_id (FK), date, check_in, check_out, hours_worked, status
leave_types	id, name (Sick/Casual/Annual/Unpaid), max_days_per_year
leave_allocations	id, employee_id, leave_type_id, allocated_days, used_days, year
leave_requests	id, employee_id, leave_type_id, start_date, end_date, reason, status, approved_by
payruns	id, month, year, status (Draft/Finalized), generated_by, generated_at
payslips	id, payrun_id, employee_id, basic, hra, allowances, gross, pf_deduction, prof_tax, net_pay, is_locked

7. API Endpoints Overview

7.1 Authentication

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register new user
POST	/api/auth/login	Public	Login and get JWT token
POST	/api/auth/logout	Auth	Invalidate session
GET	/api/auth/me	Auth	Get current user profile

Method	Endpoint	Access	Description
--------	----------	--------	-------------

POST	/api/leaves/apply	Employee+	Apply for leave
------	-------------------	-----------	-----------------

7.2 Employees

Method	Endpoint	Access	Description
GET	/api/employees	Admin, HR, Payroll	List all employees
GET	/api/employees/:id	All Roles	Get employee by ID
POST	/api/employees	Admin, HR	Create employee profile
PUT	/api/employees/:id	Admin, HR	Update employee profile
DELETE	/api/employees/:id	Admin	Delete employee

7.3 Attendance

Method	Endpoint	Access	Description
POST	/api/attendance/checkin	Employee+	Mark check-in
PUT	/api/attendance/checkout	Employee+	Mark check-out
GET	/api/attendance/my	Employee+	Own attendance logs
GET	/api/attendance/:empld	Admin, HR, Payroll	Employee attendance by ID
GET	/api/attendance/monthly	Admin, HR, Payroll	All employees monthly summary

7.4 Leave Management

GET	/api/leaves/my	Employee+	Own leave requests
GET	/api/leaves/all	Admin, HR, Payroll	All leave requests
PUT	/api/leaves/:id/approve	Admin, Payroll	Approve leave
PUT	/api/leaves/:id/reject	Admin, Payroll	Reject leave

POST	/api/leaves/allocate	Admin, HR	Allocate leave to employee
GET	/api/leaves/balance/:empId	Employee+	Leave balance by type

7.5 Payroll

Method	Endpoint	Access	Description
POST	/api/payroll/run	Admin, Payroll	Generate monthly payrun
GET	/api/payroll/runs	Admin, Payroll	List all payruns
GET	/api/payroll/payslip/:empId/:runId	Admin, Payroll, Own	Get payslip for employee
PUT	/api/payroll/payslip/:id	Admin, Payroll	Edit payslip (pre-finalize)
POST	/api/payroll/finalize/:runId	Admin, Payroll	Finalize and lock payrun
GET	/api/payroll/report	Admin, Payroll	Generate payroll report PDF

8. Project Timeline & Milestones

Phase	Tasks	Owner(s)	Goal
Phase 1	Project setup, DB schema, Auth APIs, Login UI	M2 + M1	Working login with roles
Phase 2	Employee CRUD, Attendance APIs + UI	M1 + M2 + M3	Attendance tracking live
Phase 3	Leave module APIs + UI, Payroll engine	M2 + M3 + M1	Leave workflow complete
Phase 4	Payslip generation, Reports, Dashboard charts	M1 + M2 + M3	Payroll end-to-end working
Phase 5	Data seeding, Testing, SRS finalization, Demo prep	All (led by M4)	Demo-ready system

8.1 Git Strategy

- main — stable, demo-ready code only.

- dev — integration branch, all features merge here first.
- feature/[name] — individual feature branches (e.g., feature/payroll-engine).
- Commit message format: [Module] Action — e.g., [Payroll] Add PF deduction calculation.

9. Glossary & Revision History

9.1 Abbreviations

Abbreviation	Full Form
HRMS	Human Resource Management System
RBAC	Role-Based Access Control
JWT	JSON Web Token
PF	Provident Fund
SRS	Software Requirements Specification
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ORM	Object Relational Mapper
UAT	User Acceptance Testing
FR	Functional Requirement

9.2 Document Revision History

Version	Date	Author	Changes
1.0	May 2026	Member 4 (Project Explainer)	Initial SRS document created